DANIYA ALI

SEC D

CT-25159

MR.MUDASSIR AHMED

DSA LAB 6

QUESTION NO 1

SOURCE CODE:

```cpp
#include <iostream>
#include <vector>
using namespace std;

void modifiedBubbleSort(vector<int>& arr) {
    int n = arr.size();
    bool swapped;
    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }

        if (!swapped) {
            break;
        }
    }
}

int main() {
    vector<int> arr = {1, 2, 3, 4, 5};
    modifiedBubbleSort(arr);
    cout << "Sorted array: ";
    for (int x : arr) cout << x << " ";
    cout << endl;
    return 0;
}
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy> cd "c:\
Sorted array: 1 2 3 4 5
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy>
```

QUESTION NO 2

SOURCE CODE:

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int maxToys(vector<int>& arr, int K) {
    sort(arr.begin(), arr.end());
    int count = 0;
    int spent = 0;

    for (int i = 0; i < arr.size(); i++) {
        if (spent + arr[i] <= K) {
            spent += arr[i];
            count++;
        } else {
            break;
        }
    }
    return count;
}

int main() {
    vector<int> arr = {1, 12, 5, 111, 200, 1000, 10};
    int K = 50;
    cout << "Maximum toys you can buy: " << maxToys(arr, K) << endl;
    return 0;
}
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy> cd
Maximum toys you can buy: 4
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy>
```

QUESTION NO 3

SOURCE CODE:

```cpp
#include <bits/stdc++.h>
using namespace std;
class Sort {
public:
    struct Metrics {
        long long comparisons = 0;
        long long swaps = 0;
    };

    static Metrics bubbleSort(vector<int> arr) {
        Metrics m;
        int n = arr.size();
        bool swapped;
        for (int i = 0; i < n - 1; i++) {
            swapped = false;
            for (int j = 0; j < n - i - 1; j++) {
                m.comparisons++;
                if (arr[j] > arr[j + 1]) {
                    swap(arr[j], arr[j + 1]);
                    m.swaps++;
                    swapped = true;
                }
            }
            if (!swapped) break;
        }
        return m;
    }

    static Metrics selectionSort(vector<int> arr) {
        Metrics m;
        int n = arr.size();
        for (int i = 0; i < n - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < n; j++) {
                m.comparisons++;
                if (arr[j] < arr[minIdx]) {
                    minIdx = j;
                }
            }
            if (minIdx != i) {
                swap(arr[i], arr[minIdx]);
                m.swaps++;
            }
        }
        return m;
```

```cpp
    }

    static Metrics insertionSort(vector<int> arr) {
        Metrics m;
        int n = arr.size();
        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;
            while (j >= 0) {
                m.comparisons++;
                if (arr[j] > key) {
                    arr[j + 1] = arr[j];
                    m.swaps++;
                    j--;
                } else {
                    break;
                }
            }
            arr[j + 1] = key;
        }
        return m;
    }

    static void performSorting(vector<int> arr, string technique) {
        if (technique == "bubble") bubbleSort(arr);
        else if (technique == "selection") selectionSort(arr);
        else if (technique == "insertion") insertionSort(arr);
        else cout << "Invalid sorting technique!" << endl;
    }

    static void analyzeRandomArray(int size) {
        vector<int> arr(size);
        for (int i = 0; i < size; i++) {
            arr[i] = rand() % 1000;
        }

        Metrics b = bubbleSort(arr);
        Metrics s = selectionSort(arr);
        Metrics i = insertionSort(arr);

        cout << "--- Sorting Analysis for Array Size " << size << " ---\n";
        cout << "Bubble Sort    -> Comparisons: " << b.comparisons << ", Swaps: " << b.swaps << endl;
```

```
87          cout << "Bubble Sort    -> Comparisons: " << b.comparisons << ", Swaps: " << b.swaps << endl;
88          cout << "Selection Sort -> Comparisons: " << s.comparisons << ", Swaps: " << s.swaps << endl;
89          cout << "Insertion Sort -> Comparisons: " << i.comparisons << ", Swaps: " << i.swaps << endl;
90
91          long long totalB = b.comparisons + b.swaps;
92          long long totalS = s.comparisons + s.swaps;
93          long long totalI = i.comparisons + i.swaps;
94
95          string best = "Bubble Sort", worst = "Bubble Sort";
96          long long minOps = totalB, maxOps = totalB;
97
98          if (totalS < minOps) { minOps = totalS; best = "Selection Sort"; }
99          if (totalS > maxOps) { maxOps = totalS; worst = "Selection Sort"; }
100
101         if (totalI < minOps) { minOps = totalI; best = "Insertion Sort"; }
102         if (totalI > maxOps) { maxOps = totalI; worst = "Insertion Sort"; }
103
104         cout << "\nBest Technique: " << best << endl;
105         cout << "Worst Technique: " << worst << endl;
106     }
107 };
108
109 int main() {
110     srand(time(0));
111     vector<int> sample = {5, 2, 9, 1, 5, 6};
112     Sort::performSorting(sample, "bubble");
113     Sort::analyzeRandomArray(100);
114     return 0;
115 }
```

OUTPUT:

```
Maximum toys you can buy: 4
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy> cd "c:\Use
--- Sorting Analysis for Array Size 100 ---
Bubble Sort    -> Comparisons: 4922, Swaps: 2472
Selection Sort -> Comparisons: 4950, Swaps: 93
Insertion Sort -> Comparisons: 2569, Swaps: 2472

Best Technique: Insertion Sort
Worst Technique: Bubble Sort
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy>
```

QUESTION NO 4

SOURCE CODE:

```cpp
#include <bits/stdc++.h>
using namespace std;
class Solution {
public:
    void flip(vector<int>& arr, int k) {
        int i = 0, j = k - 1;
        while (i < j) {
            swap(arr[i], arr[j]);
            i++;
            j--;
        }
    }

    vector<int> pancakeSort(vector<int>& arr) {
        vector<int> res;
        int n = arr.size();
        for (int currSize = n; currSize > 1; currSize--) {
            int maxIdx = 0;
            for (int i = 1; i < currSize; i++) {
                if (arr[i] > arr[maxIdx]) {
                    maxIdx = i;
                }
            }

            if (maxIdx == currSize - 1) continue;

            if (maxIdx != 0) {
                res.push_back(maxIdx + 1);
                flip(arr, maxIdx + 1);
            }

            res.push_back(currSize);
            flip(arr, currSize);
        }
        return res;
    }
};

int main() {
    vector<int> arr = {3, 2, 4, 1};
    Solution sol;
    vector<int> flips = sol.pancakeSort(arr);
    cout << "K-values sequence: ";
    for (int k : flips) cout << k << " ";
    return 0;
}
```

OUTPUT:

```
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy> cd "c:\Users\
K-values sequence: 3 4 2 3 2
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy>
```

QUESTION NO 5

SOURCE CODE:

Greedy > sort_colors.cpp

```cpp
#include <iostream>
#include <vector>
using namespace std;

void sortColors(vector<int>& nums) {
    int low = 0, mid = 0, high = nums.size() - 1;
    while (mid <= high) {
        if (nums[mid] == 0) {
            swap(nums[low], nums[mid]);
            low++;
            mid++;
        } else if (nums[mid] == 1) {
            mid++;
        } else {
            swap(nums[mid], nums[high]);
            high--;
        }
    }
}

int main() {
    vector<int> nums = {2, 0, 2, 1, 1, 0};
    sortColors(nums);
    cout << "Sorted colors: ";
    for (int x : nums) cout << x << " ";
    cout << endl;
    return 0;
}
```

OUTPUT:

```
K-values sequence: 3 4 2 3 2
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy> cd "
Sorted colors: 0 0 1 1 2 2
PS C:\Users\User\OneDrive\Desktop\DSA\Greedy>
```